

Class 11: Python — First Contact with the scikit-learn API

Data-driven Systems Engineering

Agenda

- The scikit-learn contract: `fit`, `predict`, and why every estimator shares it.
- Splitting data with `train_test_split`.
- Training an estimator, and searching over one with `GridSearchCV`.
- The Pipeline pattern for chaining steps.
- What this class deliberately leaves for the Data-Driven block.

Learning goals

- Call `train_test_split` correctly and explain its main arguments.
- Instantiate a scikit-learn estimator and call `.fit()` and `.predict()` on it.
- Recognize `GridSearchCV` as an object with the same `fit` contract as any estimator.
- Describe what a `Pipeline` chains together and why.
- State clearly which questions are programming mechanics today, and which are modeling theory for later.

Course Map and Running Cases



Today in the course

Focus: The scikit-learn API as programming mechanics: fit, predict, and pipeline, not model theory.

Bridge: Class 8 showed that `RandomForestClassifier` and `StandardScaler` are just classes; today is about the shared calling convention every scikit-learn class honors.

Fraud case

The fraud model in 'streamlit/modeling.py' is trained with exactly the same three calls this class walks through: split, fit, predict.

Pasteurization case

The pasteurization case will eventually reuse the same estimator contract for a forecasting or anomaly-detection model, whatever the algorithm turns out to be.

The scikit-learn contract

- Every estimator is instantiated the same way: `Estimator(hyperparameters)`.
- Every estimator is trained the same way: `estimator.fit(X, y)`.
- Every estimator predicts the same way: `estimator.predict(X)`.
- This uniform contract is a deliberate design choice, not an accident — it is what let Class 8's inheritance discussion matter in practice.

Why it matters

Once you can call `.fit()` and `.predict()` on one estimator, you can call it on almost any other, without relearning an API.

Loading data and splitting it

```
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name='target')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

- From 'notebooks/Class6_Model_Training_.ipynb': one function call turns one dataset into four pieces.
- This is pure mechanics — *why* we split before training is a modeling-theory question for the Data-Driven block.

What `train_test_split`'s arguments do

Argument	Mechanical effect
<code>test_size</code>	Fraction of rows set aside as <code>X_test</code> / <code>y_test</code> .
<code>random_state</code>	Fixes the random shuffle so the split is reproducible.
<code>stratify</code>	Splits so each subset keeps the same class proportions as <code>y</code> .

- Notice all three arguments only control *how the split happens*, not whether it is the right thing to do — that judgment call belongs to a later class.

Instantiate, then fit

```
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
```

- Three lines: build the object, train it in place, ask it for predictions on unseen rows.
- 'streamlit/modeling.py' follows the identical shape with a scaler in between:
scaler.fit_transform(X_train) then cls.fit(X_train_scaled, y_train).

GridSearchCV: still just .fit()

```
param_grid = {  
    'n_estimators': [50, 100, 200],  
    'max_depth': [3, 5, 8, None],  
    'min_samples_split': [2, 4, 6]  
}  
  
grid = GridSearchCV(rf, param_grid, cv=5, scoring='accuracy', n_jobs=-1)  
grid.fit(X_train, y_train)
```

- From the same notebook: GridSearchCV wraps an estimator and a grid of hyperparameters, but is itself called with .fit(), exactly like rf above.
- What cv and scoring actually optimize for is a modeling-theory question we postpone; today, notice only that the calling convention never changes.

What comes back out

```
grid.best_params_  
grid.best_score_  
best_model = grid.best_estimator_  
  
y_pred = best_model.predict(X_test)
```

- After `.fit()`, `grid` carries fitted attributes: the winning parameters, its score, and the trained estimator itself.
- `best_model` is just another estimator — `.predict()` works on it exactly as it did on `rf`.

The Pipeline pattern

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", RandomForestClassifier(random_state=42)),
])
pipe.fit(X_train, y_train)
pipe.predict(X_test)
```

- This is the idiomatic scikit-learn way to chain the scaler and the model into one object — not yet how 'modeling.py' is written, which fits the scaler and the model as two separate steps.
- A Pipeline is itself an estimator: it honors the same `.fit()` / `.predict()` contract as its individual pieces.

Why the uniform API is worth learning on purpose

- Swapping `RandomForestClassifier` for a different classifier changes one line, not the surrounding pipeline.
- A `GridSearchCV`-wrapped estimator, or a `Pipeline`-wrapped one, both still expose `.fit()` and `.predict()`.
- This is Class 8's inheritance discussion paying off directly: shared base classes are what make this consistency possible.

What today deliberately does not cover

- Why accuracy can mislead on an imbalanced target like fraud detection.
- How to choose `cv`, `scoring`, or a hyperparameter grid.
- Feature scaling and encoding theory — only the mechanical call, `.fit_transform()`, was shown.
- All of the above belong to the Data-Driven block, once the programming mechanics here are second nature.

How the repo already uses today's building blocks

Repo activity

- 'notebooks/Class6_Model_Training_.ipynb' runs `train_test_split`, a plain `RandomForestClassifier`, and a `GridSearchCV` over it, in that order.
- 'streamlit/modeling.py' shows the two-step, non-Pipeline version of scaling and fitting.
- Rewrite 'modeling.py's scaler-then-classifier steps as a single Pipeline and check that `.predict()` still works.

Question

'GridSearchCV' and 'Pipeline' both wrap other estimators, yet both expose the same `.fit()` / `.predict()` interface as a plain classifier. What design principle from Class 8 makes that possible?

- Ties directly back to inheritance and shared base classes, rather than treating the API's consistency as a coincidence.

Papers and materials

Buitinck et al. (2013), API design for machine learning software; Pedregosa et al. (2011), scikit-learn: Machine Learning in Python; scikit-learn pipelines and composite estimators; GridSearchCV API.

From This Class to the Next

What stays with us

- `fit`, `predict`, and the constructor call are the three moves that carry you through almost any scikit-learn estimator.
- `GridSearchCV` and `Pipeline` are themselves estimators — the API does not change as objects wrap other objects.
- Metrics, scoring choices, and hyperparameter theory are intentionally still ahead of us.

Project use

Any project step described as “train a scikit-learn model” is exactly today’s three-call shape, whatever the model turns out to be.

Next connection

Class 12 is a guided lab: applying this course’s full EDA workflow to a fresh dataset, hands-on.

Course thread: Mechanics without practice fade fast — the next session is where you drive.